Lecture 14: Beyond Supervised Learning

Kaicheng YU

SoE, Westlake University

May 19, 2025

Table of Contents

1. Data Augmentation
2. Transfer Learning
   - Fine-tuning and Supervised Pre-training
   - Unsupervised Pre-training (Self-Supervised Learning)
3. Semi-Supervised Learning
4. Active Learning
5. Meta-Learning
6. Few-Shot Learning
7. Weakly Supervised Learning
8. Conclusion

Data Augmentation: Introduction

- Many ML models (e.g., ResNet, Transformers) have more parameters than labeled training examples.
- Large models are slow to train and can easily overfit, especially with small labeled datasets.
- Data augmentation is a technique to address this by creating artificially modified versions of input data while keeping labels unchanged.
- Goal: Capture expected variations at test time.

Data Augmentation: Why?

- Suppose we have a single small labeled dataset.
- We can create artificially modified versions of input vectors.
- These modifications should capture variations expected at test time.
- Crucially, the original labels remain unchanged.
- This process helps in scenarios with limited labeled data.

Examples of Data Augmentation (Images)

- Standard methods for image classification:
  - Random crops
  - Zooms
  - Mirror image flips
- Example: Figure 19.1 illustrates random crops and zooms.

Figure: Illustration of random crops and zooms. (Fig 19.1 from source)

More Image Augmentation Examples

- Sophisticated example (GVZ16): Rendering text characters onto images realistically.
  - Created a very large dataset of text *in the wild*.
  - Used to train state-of-the-art visual text localization and reading systems.

Data Augmentation Beyond Images

- Speech: Artificially adding background noise to clean speech signals.
- Text (NLP): Artificially replacing characters or words at random in text documents.
- For more NLP augmentation examples, see Feng et al. (2021).

Automating Augmentation: AutoAugment

- If training/testing multiple times is feasible, we can learn optimal augmentations.
- Methods:
  - Reinforcement Learning (e.g., Cubuk et al., 2019)
  - Bayesian Optimization (e.g., Lim et al., 2019)
- This approach is called AutoAugment.
- Can also learn to combine multiple augmentations (also AutoAugment, Cubuk et al., 2019).

Theoretical Justification for Data Augmentation

- Data augmentation often significantly improves performance (accuracy, robustness).
- It might seem like getting something for nothing (no new external data).
- However, data augmentation is a way to algorithmically inject **prior knowledge**.

Theoretical Justification: ERM Perspective

- Standard Empirical Risk Minimization (ERM): Minimize empirical risk $R(f) = \int \ell(f(\boldsymbol{x}), \boldsymbol{y}) p^*(\boldsymbol{x}, \boldsymbol{y}) d\boldsymbol{x} d\boldsymbol{y}$.
- $p^*(x, y)$ is approximated by the empirical distribution:

$$p_{\mathcal{D}}(\boldsymbol{x}, \boldsymbol{y}) = \frac{1}{N} \sum_{n=1}^{N} \delta(\boldsymbol{x} - \boldsymbol{x}_n) \delta(\boldsymbol{y} - \boldsymbol{y}_n)$$

- Data augmentation replaces empirical distribution with an algorithmically smoothed one:

$$p_{\mathcal{D}}(\boldsymbol{x}, \boldsymbol{y} | A) = \frac{1}{N} \sum_{n=1}^{N} p(\boldsymbol{x} | \boldsymbol{x}_n, A) \delta(\boldsymbol{y} - \boldsymbol{y}_n)$$

- A is the augmentation algorithm, generating $\tilde{\boldsymbol{x}}$ from $\boldsymbol{x}_n$ such that label is unchanged.

Theoretical Justification: Vicinal Risk Minimization

- The smoothed distribution $p_{\mathcal{D}}(\boldsymbol{x}, \boldsymbol{y}|A)$ means we are minimizing risk in the **vicinity** of each training point $\boldsymbol{x}_n$.
- Example of $p(\boldsymbol{x}|\boldsymbol{x}_n, A)$: A Gaussian kernel, $\mathcal{N}(\boldsymbol{x}|\boldsymbol{x}_n, \sigma^2\mathbf{I})$.
- This concept is called Vicinal Risk Minimization (Chapelle et al., 2001).
- For more details: Zhang et al. (2017b), Dao et al. (2019).

Transfer Learning: Introduction

- Many data-poor tasks share structural similarity with data-rich tasks.
- Example: Classifying endangered (rare) birds vs. common birds.
  - Limited images for endangered species.
  - Common bird species have abundant data.
  - Structural similarities: wings, feathers, beaks, etc.
- Idea: Train on a large dataset (non-endangered birds) then continue training (fine-tune) on the small dataset (endangered species).
- This is called **transfer learning**: transferring information from one dataset to another via shared parameters.

Transfer Learning: Pre-training and Fine-tuning

- Two main phases:
  1. **Pre-training phase**: Train a model with parameters $\boldsymbol{\theta}$ on a large source dataset $\mathcal{D}_p$.
     - Source dataset can be labeled or unlabeled.
  2. **Fine-tuning phase**: Adapt the model on a small labeled target dataset $\mathcal{D}_q$ of interest.
- For more information, see surveys like Tan et al. (2018), Zhu et al. (2021).
- Figure 19.2 illustrates fine-tuning.

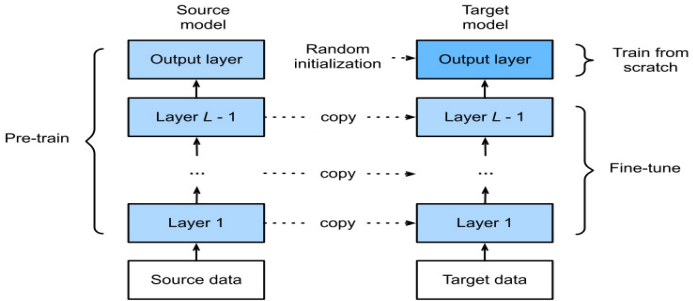


Figure: Fine-tuning a model on a new dataset. (Fig 19.2 from source)

Fine-tuning: The Process

- Assume a pretrained classifier $p(y|x, \boldsymbol{\theta}_p)$ (e.g., CNN on ImageNet).
- Works well for source inputs $\boldsymbol{x} \in \mathcal{X}_p$ and outputs $y \in \mathcal{Y}_p$.
- Goal: Create a new model $q(y|\boldsymbol{x}, \boldsymbol{\theta}_q)$ for target inputs $\boldsymbol{x} \in \mathcal{X}_q$ and outputs $y \in \mathcal{Y}_q$.
- Often, input domains are similar ($\mathcal{X}_q \approx \mathcal{X}_p$, e.g., both RGB images).
- If input domains differ significantly, may need *domain adaptation* (later topic).

Fine-tuning: Handling Different Output Domains

- Output domains $\mathcal{Y}_q$ and $\mathcal{Y}_p$ are usually different.
- Example: $\mathcal{Y}_p$ = ImageNet labels, $\mathcal{Y}_q$ = medical image labels.
- Neural Network Solution: **Model Surgery**
  - "Chop off" the final layer of the original model.
  - Add a new "head" (output layer) for the new class labels.
  - Illustrated in Figure 19.2.
- Example: If $p(\boldsymbol{y}|\boldsymbol{x}, \boldsymbol{\theta}_p) = \mathrm{softmax}(\boldsymbol{y}|\mathbf{W}_2\boldsymbol{h}(\boldsymbol{x}; \boldsymbol{\theta}_1) + \boldsymbol{b}_2)$.
- New model: $q(y|\boldsymbol{\theta}_q) = \mathrm{softmax}(y|\mathbf{W}_3\boldsymbol{h}(\boldsymbol{x}; \boldsymbol{\theta}_1) + \boldsymbol{b}_3)$.
- $\boldsymbol{h}(\boldsymbol{x}; \boldsymbol{\theta}_1)$ is the shared nonlinear feature extractor.

Fine-tuning: Parameter Updates

- After model surgery, fine-tune the new model with parameters $\boldsymbol{\theta}_q = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_3)$.
  - $\boldsymbol{\theta}_1$: parameters of the feature extractor.
  - $\boldsymbol{\theta}_3$: parameters of the new final linear layer.
- Option 1: Freeze feature extractor parameters $\boldsymbol{\theta}_1$.
  - Only train $\boldsymbol{\theta}_3$ (linear classifier on extracted features).
  - Leads to a convex optimization problem.
  - Helpful in long-tail settings (some classes are rare).
- Option 2: Fine-tune all parameters $(\boldsymbol{\theta}_1, \boldsymbol{\theta}_3)$.
  - Allow $\boldsymbol{\theta}_1$ to be updated, but often with a **lower learning rate**.
  - Prevents moving too far from values learned on source dataset $\mathcal{D}_p$.

Adapters: An Alternative to Full Fine-tuning

- Disadvantage of full fine-tuning:
  - Can be slow (many parameters).
  - Small learning rate needed to avoid divergence of low-level features.
  - Each new task requires a new, full model copy.
- **Adapters**: Keep pre-trained model untouched; add new parameters to modify internal behavior for each task.
- Customizes feature extraction per task. (e.g., [RBV17, RBV18, Hou+19])

Adapters: Examples

- **Transformer Adapters** (Hou+19):
  - Insert shallow bottleneck MLPs inside each Transformer layer (after MHA, after FFN).
  - Skip connections allow initialization to identity.
  - Introduces $O(DM)$ new parameters per layer (D=feat dim, M=bottleneck size).
  - Only adapter MLPs, layer norms, and output head are trained.
  - Empirically better performance than fine-tuning on some NLP tasks, with 1-10
- **ResNet Adapters** (RBV17, RBV18): Add 1x1 conv layers (α) to internal CNN layers.
- Can be added in series or parallel.

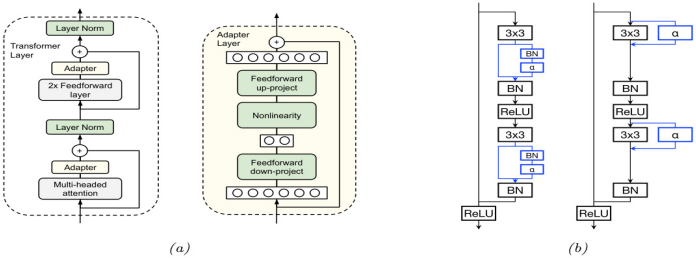


Figure: (a) Transformer Adapters. (b) ResNet Adapters. (Fig 19.3 from source)

Supervised Pre-training

- Pre-training task can be supervised or unsupervised.
- Key requirements:
  - Teaches model basic structure of the problem domain.
  - Sufficiently similar to the downstream fine-tuning task.
- Task similarity: Not rigorously defined; often pre-training domain is broader than fine-tuning domain (e.g., all birds vs. endangered birds).

Supervised Pre-training: ImageNet Example

- Most straightforward: Large labeled dataset available for pre-training.
- Common example: ImageNet dataset for pre-training CNNs.
  - 1.28 million natural images, 1000 classes.
  - Diverse concepts: animals, food, buildings, etc.
  - Diverse images: various angles, sizes, backgrounds.
- This diversity and scale likely why ImageNet is a de-facto pre-training standard in computer vision.

Supervised Pre-training: Limitations & Non-Vision

- **Limitations**:
  - ImageNet pre-training less helpful if fine-tuning task domain is very different (e.g., medical images vs. natural images [Rag+19]).
  - For some tasks (e.g., object detection), might be more of a speedup (warm-start) than essential for performance if trained from scratch long enough [HGD19].
- **Non-Vision Applications**:
  - Less common but exists.
  - NLP: Pre-train on natural language inference (NLI) data for sentence representations [Con+17] (largely supplanted by unsupervised methods).
  - Speech: Pre-train speech recognition on large English corpus, fine-tune on low-resource languages [Ard+20].

Unsupervised Pre-training (Self-Supervised Learning)

- Increasingly common due to ease of acquiring unlabeled data (e.g., web images, text).
- Differs from older unsupervised pre-training (e.g., autoencoders on the labeled set before supervised training).
- Modern context: Uses a **large unlabeled dataset** for pre-training, then fine-tunes on a **different, smaller, labeled dataset**.
- Often called **self-supervised learning (SSL)**:
  - Labels are created by the algorithm itself (not by humans).
  - Both supervised and SSL are discriminative (predict outputs given inputs).
  - Contrasts with generative unsupervised methods (predict outputs unconditionally, Ch. 20).

Self-Supervised Learning: Main Approaches

- Many SSL heuristics exist (see reviews [GR18, JT19, Ren19]).
- Three main broad groups:
  1. **Imputation Tasks**:
     - Partition input $\boldsymbol{x} = (\boldsymbol{x}_h, \boldsymbol{x}_v)$.
     - Predict hidden part $\boldsymbol{x}_h$ from visible part $\boldsymbol{x}_v$.
     - "Fill-in-the-blank" or "cloze task" (NLP).
     - Visual examples in Figure 19.4.
  2. **Proxy Tasks (Pretext Tasks)**:
     - Create input pairs (x_1, x_2).
     - Train a Siamese network $p(y|f(x_1), f(x_2))$ to predict a label y capturing relationship between x_1, x_2.
     - Example: x_1=image patch, x_2=rotated x_1, y=rotation angle.
  3. **Contrastive Tasks**: Current most popular.

SSL: Contrastive Tasks

- Basic Idea:
  - Create pairs of semantically similar examples using data augmentation (Sec 19.1).
  - Ensure their representations are closer (in embedding space) than representations of unrelated examples.
- Same core idea as Deep Metric Learning (Sec 16.2.2).
- Key difference: Algorithm creates its own similar pairs, not relying on external labels.
- Examples: SimCLR, CLIP (discussed next).

SSL: SimCLR - Introduction

- SimCLR: Simple Contrastive Learning of Visual Representations [Che+20b, Che+20c].
- State-of-the-art performance in transfer learning and semi-supervised learning.
- Basic Idea:
  - Input $\boldsymbol{x}$ converted to two augmented "views": $\boldsymbol{x}_1 = t_1(\boldsymbol{x})$, $\boldsymbol{x}_2 = t_2(\boldsymbol{x})$.
  - t_1, t_2 are semantics-preserving transformations (e.g., random crops, color distortion).
  - Sample "negative" examples $\boldsymbol{x}_i^-$ (semantically different, e.g., other images in minibatch).
  - Define feature mapping $F : \mathbb{R}^D \to \mathbb{R}^E$.

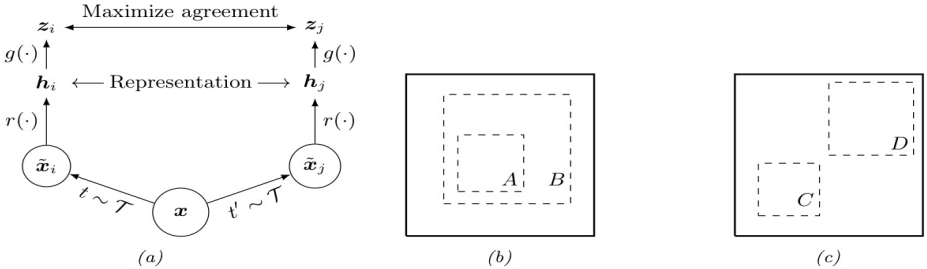


Figure: SimCLR Training Illustration (Fig 19.5a from source)

SSL: SimCLR - Objective and Details

- Objective: Maximize similarity of similar views, minimize similarity of different views.
- Loss for input $\boldsymbol{x}$ (simplified):

$$J = F(t_1(\boldsymbol{x}))^T F(t_2(\boldsymbol{x})) - \log \sum_{\boldsymbol{x}_i^- \in N(\boldsymbol{x})} \exp[F(\boldsymbol{x}_i^-)^T F(t_1(\boldsymbol{x}))]$$

- Uses cosine similarity (so F outputs are ℓ_2-normalized).
- Intermediate representation $\boldsymbol{h} = r(\boldsymbol{x})$ is used for fine-tuning, $F(\boldsymbol{x}) = g(r(\boldsymbol{x}))$.
- Critical: Choice of data augmentations (random cropping, flips, color distortion).
- Relies on large batch training for diverse negatives.
- Alternative for smaller batches: Memory bank (MoCo - Momentum Contrastive Learning [He+20]).

SSL: CLIP - Introduction

- CLIP: Contrastive Language-Image Pre-training [Rad+].
- Uses massive corpus of 400M (image, text) pairs from the web.
- Let $\boldsymbol{x}_i$ be i-th image, $\boldsymbol{y}_i$ its matching text.
- Goal: Determine if $\boldsymbol{y}_i$ is more likely correct for $\boldsymbol{x}_i$ than some $\boldsymbol{y}_j$, and vice-versa.
- Embeddings: Image $f_I(\boldsymbol{x}_i)$, Text $f_T(\boldsymbol{y}_j)$.
- Normalized embeddings: $\mathbf{I}_i = f_I(\boldsymbol{x}_i)/||f_I(\boldsymbol{x}_i)||_2$, $\mathbf{T}_j = f_T(\boldsymbol{y}_j)/||f_T(\boldsymbol{y}_j)||_2$.
- Logits: $L_{ij} = \mathbf{I}_i^T \mathbf{T}_j$.

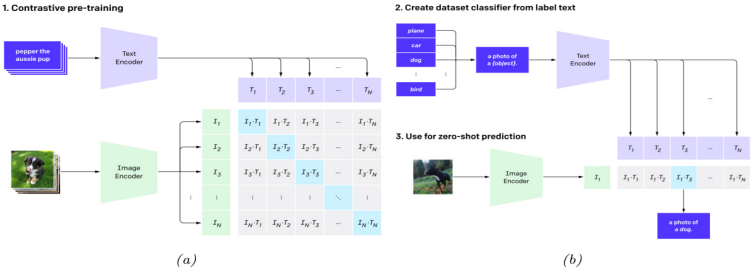


Figure: CLIP Model Illustration (Fig 19.7 from source)

SSL: CLIP - Loss and Zero-Shot Classification

- Loss (minimized over minibatches of size N):

$$J = \frac{1}{2}\left[\sum_{i=1}^{N} \mathrm{CE}(\mathbf{L}_{i,:}, \mathbf{1}_i) + \sum_{j=1}^{N} \mathrm{CE}(\mathbf{L}_{:,j}, \mathbf{1}_j)\right]$$

- CE is Cross Entropy loss; $\mathbf{1}_i$ is one-hot encoding of label i.
- Scaled by a learned temperature parameter in practice.
- Models: ResNet/ViT for f_I, Text Transformer for f_T. Large minibatches ($N \sim 32k$).
- **Zero-Shot Classification**:
  1. Convert K class labels to text strings (e.g., "dog" $\rightarrow$ "a photo of a dog"). This is *prompt engineering*.
  2. Compute embeddings $\mathbf{I} \propto f_I(\text{new image})$ and $\mathbf{T}_k \propto f_T(\text{text for class } k)$.
  3. Compute softmax: $p(y = k|\text{image}) = \mathrm{softmax}([\mathbf{I}^T\mathbf{T}_1, \ldots, \mathbf{I}^T\mathbf{T}_K])_k$.
- Remarkable performance, even vs. supervised on ImageNet, robust to distribution shift.
- Drawback: Sensitive to prompt engineering.

Semi-Supervised Learning (SSL): Motivation

- Many successful ML applications are supervised, requiring large labeled datasets.
- However, obtaining labeled data can be expensive and time-consuming.
- Example: Speech recognition - transcribing audio is slow.
- Example: Medical applications - requires expert (e.g., doctor) labeling, increasing cost.
- Semi-supervised learning aims to alleviate this by using **unlabeled data**.

Semi-Supervised Learning: Goal and Assumptions

- General Goal: Allow the model to learn high-level data structure from unlabeled data, relying on labeled data for fine-grained task details.
- Standard Supervised Learning: Access to samples from joint distribution $p(\boldsymbol{x}, y)$.
- Semi-Supervised Learning: Additionally access samples from marginal distribution $p(\boldsymbol{x})$ (unlabeled data).
- Assumption: Many more unlabeled samples are available as they are cheaper to obtain.

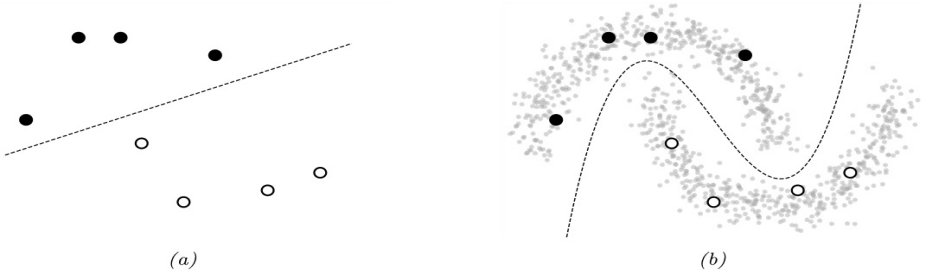


Figure: Benefits of SSL. (a) Boundary with labeled data only. (b) Boundary with labeled and unlabeled data.

SSL: Self-Training / Pseudo-Labeling

- Early and straightforward SSL approach [Scu65, McL75]. Also "pseudo-labeling" [Lee13].
- Basic Idea: Use the model itself to infer predictions (pseudo-labels) on unlabeled data, then treat these as labels for re-training.
- Applicable to any model that can generate predictions.
- Two main procedures:
  - **Offline**: Pseudo-label all unlabeled data, re-train model to convergence. Repeat. (Good for giant unlabeled datasets [Yal+19, Xie+20])
  - **Online**: Continually generate pseudo-labels on batches of unlabeled data and immediately train. (Often a component in sophisticated SSL methods [Soh+20])

Self-Training: Confirmation Bias & Mitigation

- **Problem: Confirmation Bias** [TV17]
  - If model generates incorrect pseudo-labels and re-trains on them, it can get progressively worse.
  - Model continually confirms its own (incorrect) bias.
- **Mitigation: Selection Metric** [RHS05]
  - Heuristically try to only retain pseudo-labels that are likely correct.
  - Example: If model outputs class probabilities, only retain pseudo-labels where the max probability is above a threshold [Yar95].
  - Assumes well-calibrated probabilities.

SSL: Entropy Minimization

- Self-training implicitly encourages low-entropy (high-confidence) predictions.
- Online self-training loss (cross-entropy on pseudo-labels):

$$\mathcal{L} = -\max_c \log p_\theta(y = c|\boldsymbol{x})$$

- Explicit Entropy Minimization loss [GB05]:

$$\mathcal{L} = -\sum_{c=1}^{C} p_\theta(y = c|\boldsymbol{x}) \log p_\theta(y = c|\boldsymbol{x})$$

- Both minimized when model assigns all probability to a single class.
- Online self-training is like entropy minimization with a "hard" target.
- MixMatch [Ber+19b] interpolates using a temperature T. ($T = 1 \rightarrow$ entropy min, $T \rightarrow 0 \rightarrow$ hard self-training).

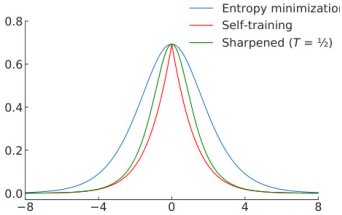
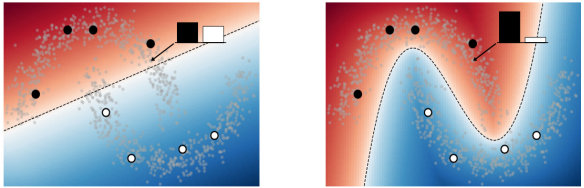


Figure: Comparison of entropy minimization, self-training, and sharpened loss. (Fig 19.9 from source)

Entropy Minimization: The Cluster Assumption

- Why is entropy minimization a good idea?
- **Cluster Assumption**: Decision boundary should fall in low-density regions.
  - Data for different classes form clusters.
  - Good boundary separates clusters, doesn't pass through them.
- SSL methods with cluster assumption use unlabeled data to estimate data manifold shape and move boundary away from dense regions.
- If boundary passes through high-density region, model produces high-entropy predictions for some samples (assuming smooth boundary).
- Entropy minimization encourages boundary in low-density regions.



(a) (b)

Figure: Entropy minimization enforcing cluster assumption. (Fig 19.10 from source)

SSL: Co-Training

- Assumes two complementary "views" (independent feature sets) of the data [BM98].
- Train two models separately on each view.
- Unlabeled data classified by both models.
- If one model is confident (low-entropy) and other is not (high-entropy) for a pseudo-label, add it to training set of low-confidence model.
- Repeat iteratively.
- **Tri-Training** [ZL05] avoids strong two-view assumption:
  - Uses three models, trained on independently sampled subsets of labeled data.
  - If two models agree on a pseudo-label for an unlabeled point, add to training set for the third model.

SSL: Label Propagation on Graphs

- Leverages the **manifold assumption**: similar datapoints likely share a label.
- Construct a graph: nodes = data examples, edge weights = similarity.
- Propagate known labels from labeled data across edges to unlabeled data.
- Basic Algorithm [ZG02]:
  - Define $(M+N) \times (M+N)$ transition matrix $\mathbf{T}$ where $\mathbf{T}_{i,j} = w_{i,j} / \sum_k w_{k,j}$.
  - Define $(M+N) \times C$ label matrix $\mathbf{Y}$ (class probabilities).
  - Iterate: $\mathbf{Y} \leftarrow \mathbf{T}\mathbf{Y}$; re-normalize rows of $\mathbf{Y}$; clamp labeled data rows to one-hot.
- Success depends heavily on similarity measure used for $w_{i,j}$.
- Transductive learning (predicts for fixed unlabeled set), but can induce a model.

SSL: Consistency Regularization

- Idea: Perturbing a datapoint (or model) should not dramatically change model output [BAP14].
- Applicable to unlabeled data as it only uses model outputs.
- General form: Input $\boldsymbol{x}$, model $p_\theta(y|x)$, stochastic transformations $q(\boldsymbol{x}'|\boldsymbol{x})$.
- Minimize loss like $||p_\theta(y|x) - p_\theta(y|x')||^2$.
- $p_\theta(y|x)$ often treated as fixed (no gradient).
- Combined loss (labeled $\mathcal{D}_L$, unlabeled $\mathcal{D}_U$):

$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_{i=1}^{M} \log p_\theta(y = y_i|\boldsymbol{x}_i) + \lambda \sum_{j=1}^{N} |p_\theta(y|\boldsymbol{x}_j) - p_\theta(y|\boldsymbol{x}'_j)|^2$$

- λ balances supervised and unsupervised loss; often ramped up during training.

Consistency Regularization: Details

- **Transformations** $q(x'|x)$: Should not change label. Strong data augmentations often used [Xie+19, Soh+20].
- **Virtual Adversarial Training (VAT)** [Miy+18]: Analytically find perturbation δ to maximally change output.

$$\delta \approx \operatorname{argmax}_\delta D_{\mathbb{KL}}\left(p_\theta(y|\boldsymbol{x}) \parallel p_\theta(y|\boldsymbol{x}+\delta)\right)$$

  Then $\boldsymbol{x}' = \boldsymbol{x} + \epsilon \frac{\delta}{|\delta|_2}$.
- **Loss Function Choice**: Squared L2 distance (Brier score) or KL divergence.
  - Squared L2: Saturates if predictions differ greatly (zero gradient).
  - KL Divergence: Same scale as supervised cross-entropy loss, easier λ tuning.

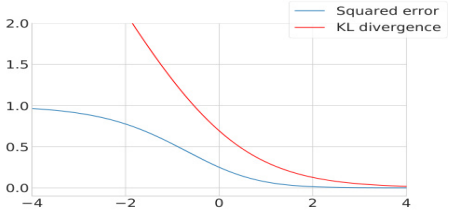


Figure: Squared error vs KL divergence for consistency regularization. (Fig 19.11 from source)

SSL with Deep Generative Models (Overview)

- Generative models naturally use unlabeled data by modeling $p_\theta(\boldsymbol{x})$.
- Goal: Use learned $p_\theta(\boldsymbol{x})$ to improve supervised model.
- Examples:
  - Variational Autoencoders (VAEs)
  - Generative Adversarial Networks (GANs)
  - Normalizing Flows
- (Details on these models often in later chapters, e.g., Chapter 20 for VAEs, or a sequel book for GANs/Flows as per text).

SSL: VAEs (M1 & M2 models [Kin+14])

- VAEs model $p(\boldsymbol{x}, \boldsymbol{z})$ where $\boldsymbol{z}$ are latent variables.
- **M1 Model**: Train VAE on unlabeled data. Use learned latent variables $\boldsymbol{z}$ (lower-dim, often more separable) as features for a downstream classifier.
- **M2 Model**: Assumes data generation $p_\theta(\boldsymbol{x}, y) = p_\theta(y) \int p_\theta(\boldsymbol{x}|y, z) p_\theta(z) dz$.
  - Class y is also a latent variable (observed for labeled, unobserved for unlabeled).
  - Uses inference networks $q_\phi(z|y, \boldsymbol{x})$ for labeled data and $q_\phi(z, y|\boldsymbol{x})$ for unlabeled data.
  - $q_\phi(y|\boldsymbol{x})$ acts as a discriminative classifier for unlabeled data.
  - Combined objective includes ELBO terms and a classification loss on supervised data.

SSL: GANs

- GANs: Generator $G(z)$ and Critic $p_\theta(y|x)$.
- Standard GANs don't directly give latent representation for given $\boldsymbol{x}$.
- SSL with GANs: Modify critic to output C class labels OR a "fake" label (total $C+1$ outputs) [Sal+16, Ode16].
- Critic Training:
  - Labeled real data: Predict correct class.
  - Unlabeled real data: Predict any of C classes (increase probability of any real class).
  - Generated data $G(z)$: Predict "fake".
- Generator Training: Fool critic into predicting a real class for $G(z)$.

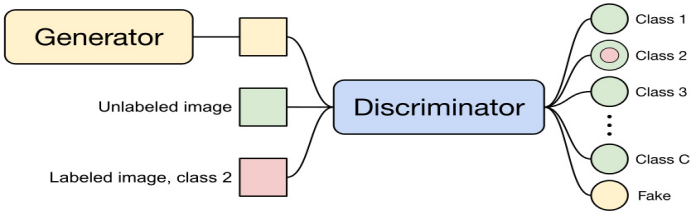


Figure: Semi-supervised GAN framework. (Fig 19.12 from source)

SSL: Normalizing Flows [Izm+20]

- Define an invertible mapping $f_\theta : \mathcal{X} \to \mathcal{Z}$ (data to latent space).
- Density in data space via change of variables: $p(x) = p(f(x)) \cdot |\det(\partial f / \partial x)|$.
- For SSL, latent distribution conditioned on label k: $p(z|y = k) = \mathcal{N}(z|\mu_k, \Sigma_k)$.
- Likelihood for labeled data $(x, y = k)$: $p(x|y = k) = \mathcal{N}(f(x)|\mu_k, \Sigma_k) \cdot |\det(\partial f / \partial x)|$.
- Likelihood for unlabeled data x: $p(x) = \sum_k p(x|y = k)p(y = k)$.
- Maximize joint likelihood of labeled and unlabeled data.
- Predictions via Bayes rule: $p(y = c|x) = p(x|y = c)p(y = c)/p(x)$.

Combining Self-Supervised and Semi-Supervised Learning [Che+20c]

- It is possible to combine SSL (representation learning) and SSL (leveraging unlabeled data for a task).
- Example Workflow (SimCLR based):
  1. Perform self-supervised representation learning (e.g., SimCLR) on unlabeled data $\mathcal{D}_U$.
  2. Fine-tune representation on a small labeled dataset $\mathcal{D}_L$ (transfer learning) to get a teacher model T.
  3. Apply the trained teacher T back to the original unlabeled dataset $\mathcal{D}_U$ to get pseudo-labels.
  4. Distill the predictions from teacher T into a student model S (often smaller) by training S on these pseudo-labels (self-training). (τ is a temperature for label smoothing)

$$\mathcal{L}(T) = -\sum_{\boldsymbol{x}_i \in \mathcal{D}} \left[\sum_y p^T(y|\boldsymbol{x}_i;\tau) \log p^S(y|\boldsymbol{x}_i;\tau) \right]$$

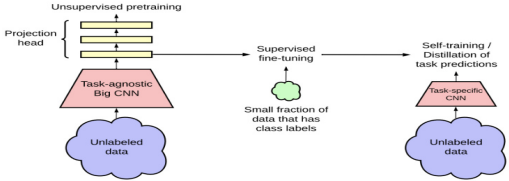


Figure: Combining SSL, supervised fine-tuning, and self-training. (Fig 19.13 from source)

Active Learning: Introduction

- Goal: Identify the true predictive mapping $y = f(\boldsymbol{x})$ by querying as few $(\boldsymbol{x}, y)$ points as possible.
- Motivation: Labeling data can be expensive or time-consuming; we want to select the most informative data points to label.
- Three main variants:
  - **Query Synthesis**: Algorithm chooses any input $\boldsymbol{x}$ and asks for its label $y = f(\boldsymbol{x})$.
  - **Pool-based Active Learning**: A large, fixed set of unlabeled data points exists. Algorithm asks for labels for one or more of these.
  - **Stream-based Active Learning**: Data arrives continuously. Algorithm decides whether to request a label for the current input.
- This section focuses on pool-based active learning (see [Set12] for a review).

Active Learning: Related Problems

- **Bayesian Optimization**:
  - Goal: Estimate the location of the global optimum $\boldsymbol{x}^* = \mathrm{argmin}_{\boldsymbol{x}} f(\boldsymbol{x})$ in as few queries as possible.
  - Typically fits a surrogate model to intermediate $(\boldsymbol{x}, y)$ queries to decide the next query.
- **Experiment Design**:
  - Goal: Infer parameters $\boldsymbol{\theta}$ of a model using carefully chosen data samples $\mathcal{D} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$.
  - Estimate $p(\boldsymbol{\theta}|\mathcal{D})$ using minimal data.
  - Can be seen as an unsupervised or generalized form of active learning.

Active Learning: Decision-Theoretic Approach

- Proposed in [KHB07, RM01].
- Defines utility of querying $\boldsymbol{x}$ as the **value of information**.
- Utility $U(\boldsymbol{x})$:

$$U(\boldsymbol{x}) \triangleq \mathbb{E}_{p(y|\boldsymbol{x},\mathcal{D})}\left[\min_{\boldsymbol{a}}\left(\rho(a|\mathcal{D}) - \rho(a|\mathcal{D},(\boldsymbol{x},y))\right)\right]$$

  where $\rho(a|\mathcal{D}) = \mathbb{E}_{p(\boldsymbol{\theta}|\mathcal{D})}[L(\boldsymbol{\theta},a)]$ is the posterior expected loss of a future action a given data $\mathcal{D}$.
- Interpretation: How much does observing (x,y) reduce the expected loss of the best future action?
- Challenge: Evaluating $U(\boldsymbol{x})$ is expensive.
  - For each possible response y to query x, must update beliefs and re-evaluate future decisions.
  - Similar to lookahead search in belief states.

Active Learning: Information-Theoretic Approach

- Avoids task-specific loss functions; focuses on learning the model parameters $\boldsymbol{\theta}$ well.
- Utility of querying $\boldsymbol{x}$ as **information gain** about $\boldsymbol{\theta}$ (reduction in entropy) [Lin56]:

$$U(\boldsymbol{x}) \triangleq \mathbb{H}(p(\boldsymbol{\theta}|\mathcal{D})) - \mathbb{E}_{p(y|\boldsymbol{x},\mathcal{D})}[\mathbb{H}(p(\boldsymbol{\theta}|\mathcal{D},\boldsymbol{x},y))]$$

- This is equivalent to the expected KL divergence (change in posterior over parameters):

$$U'(\boldsymbol{x}) \triangleq \mathbb{E}_{p(y|\boldsymbol{x},\mathcal{D})}[D_{\mathbb{KL}}(p(\boldsymbol{\theta}|\mathcal{D},\boldsymbol{x},y) \parallel p(\boldsymbol{\theta}|\mathcal{D}))]$$

Information-Theoretic Approach: BALD

- Using symmetry of mutual information, the utility $U(\boldsymbol{x})$ can be rewritten as:

$$U(\boldsymbol{x}) = \mathbb{I}(\boldsymbol{\theta}, \boldsymbol{y}|\mathcal{D}, \boldsymbol{x}) = \mathbb{H}(p(\boldsymbol{y}|\boldsymbol{x}, \mathcal{D})) - \mathbb{E}_{p(\boldsymbol{\theta}|\mathcal{D})}[|\mathbb{H}(p(\boldsymbol{y}||\boldsymbol{x}, \boldsymbol{\theta}))]$$

- Advantage: Reason about uncertainty of predictive distribution $p(y|x, \mathcal{D})$ instead of $p(\theta|\mathcal{D})$.
- Interpretation:
  - First term $\mathbb{H}(p(y|x, \mathcal{D}))$: Prefers examples $\boldsymbol{x}$ where predicted label is uncertain (Maximum Entropy Sampling [SW87]).
  - Second term $-\mathbb{E}_{p(\theta|\mathcal{D})}[|\mathbb{H}(p(y|x, \boldsymbol{\theta}))]$: Prefers examples where label is certain *once $\boldsymbol{\theta}$ is known*. Avoids inherently ambiguous/mislabeled points.
- Selects examples for which the model makes confident predictions that are highly diverse.
- Called **Bayesian Active Learning by Disagreement (BALD)** [Hou+12].
- Used for medical images, astronomical images [Wal+20].

Active Learning: Batch Active Learning

- So far: Greedy/myopic strategy (select one example $\boldsymbol{x}$ at a time).
- What if we have a budget to collect a set of B samples $(\mathbf{X}, \mathbf{Y})$?
- Information gain becomes: $U(\mathbf{X}) = \mathbb{H}(p(\boldsymbol{\theta}|\mathcal{D})) - \mathbb{E}_{p(\mathbf{Y}|\mathbf{X},\mathcal{D})}[\mathbb{H}(p(\boldsymbol{\theta}|\mathcal{D}, \mathbf{X}, \mathbf{Y}))]$.
- Optimizing this is NP-hard in horizon B [KLQ95, KG05].

Batch Active Learning: BatchBALD

- Define $f(\mathbf{Y}) \triangleq \mathbb{H}(p(\boldsymbol{\theta}|\mathcal{D})) - \mathbb{H}(p(\boldsymbol{\theta}|\mathcal{D}, \mathbf{X}, \mathbf{Y}))$ as the information gain for a batch.
- $f(\emptyset) = 0, f$ is non-decreasing (more information never hurts).
- Proving f is sub-modular [KG05].
- Consequence: Sequential greedy approach is within a constant factor of optimal for sub-modular functions.
- Combining greedy selection with the BALD objective gives **BatchBALD** [KAG19].

Meta-Learning: Learning to Learn

- Think of a learning algorithm A as mapping data $\mathcal{D}$ to parameters $\boldsymbol{\theta} = A(\mathcal{D})$.
- Algorithm A often has its own hyper-parameters ϕ (e.g., learning rate, initial $\boldsymbol{\theta}$ values). So, $\boldsymbol{\theta} = A(\mathcal{D}; \phi)$.
- **Meta-Learning Goal**: Learn the hyper-parameters ϕ themselves.
- Given a collection of datasets $\mathcal{D}_{1:J}$ and a meta-learning algorithm M, learn $\phi = M(\mathcal{D}_{1:J})$.
- Then, apply $A(\cdot; \phi)$ to learn parameters $\boldsymbol{\theta}_{J+1}$ on a new dataset $\mathcal{D}_{J+1}$.
- Also called "learning to learn" [TP97].
- See reviews [Van18, HRP21].

Model-Agnostic Meta-Learning (MAML) [FAL17]

- Natural approach: Hierarchical Bayesian Model (see Figure 19.14).
  - Parameters for each task j, $\boldsymbol{\theta}_j$, drawn from a common prior $p(\boldsymbol{\theta}_j|\boldsymbol{\xi})$.
  - Prior parameters $\boldsymbol{\xi}$ (equivalent to ϕ) are learned (meta-learning).
  - Pools statistical strength from multiple data-poor problems.

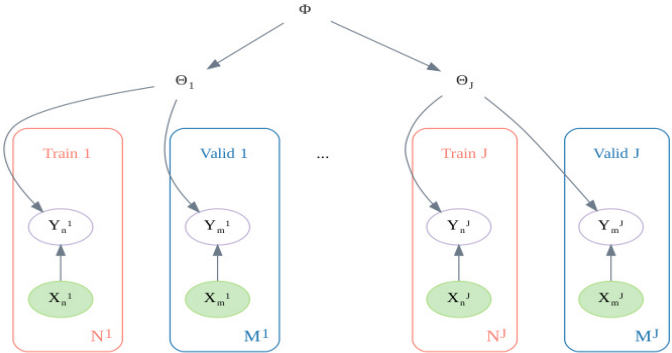


Figure: Hierarchical Bayesian model for meta-learning. (Fig 19.14 from source)

MAML: Empirical Bayes Approach

- Instead of full Bayesian inference, MAML uses an Empirical Bayes approximation:

$$\boldsymbol{\xi}^* = \arg\max_{\boldsymbol{\xi}} \frac{1}{J} \sum_{j=1}^{J} \log p\big(\mathcal{D}^j_{\text{valid}} | \hat{\boldsymbol{\theta}}_j(\boldsymbol{\xi}, \mathcal{D}^j_{\text{train}})\big),$$

  where $\hat{\boldsymbol{\theta}}_j$ is a point estimate for task j based on its training data $\mathcal{D}^j_{\text{train}}$ and prior $\boldsymbol{\xi}$.
- Uses cross-validation approximation to marginal likelihood.
- To compute point estimate for a new target task $\hat{\boldsymbol{\theta}}_{J+1}$:
  - Start at $\boldsymbol{\xi}^*$ (the learned prior/meta-parameters).
  - Perform K steps of gradient ascent with learning rate η.
- This is MAML: Model-Agnostic Meta-Learning.
- Equivalent to approximate MAP estimate with a Gaussian prior centered at $\boldsymbol{\xi}$ (strength controlled by K) [San96, Gra+18]. This is fast adaptation.

Few-Shot Learning (FSL): Introduction

- Humans can learn to predict from very few labeled examples.
- **Few-Shot Learning (FSL)**: Learning to classify C classes using just N training examples of each class (C-way N-shot classification).
- Typically N and C are very small.
- **One-Shot Learning**: $N = 1$.
- **Zero-Shot Learning**: $N = 0$ (no labeled examples for the target classes, relies on other information like attributes or text descriptions - CLIP is an example).
- Since data for the new domain/task is so small, learning from scratch is often infeasible. Meta-learning is a common approach.

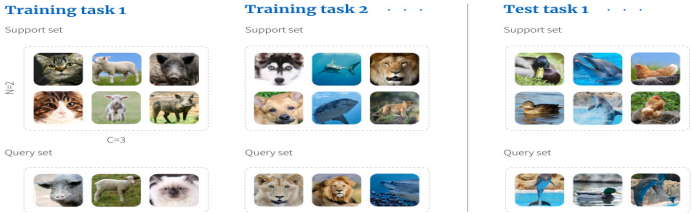


Figure: Illustration of a 3-way 2-shot problem setup for meta-learning. (Fig 19.15 from source)

FSL: Meta-Learning Setup

- During meta-training:
  - Meta-algorithm M trains on a labeled **support set** from task j.
  - Returns a predictor f^j.
  - f^j is evaluated on a disjoint **query set** also from task j.
  - M is optimized over all J training tasks.
- During meta-testing (on a new task):
  - Apply M to the new labeled support set to get f^{test}.
  - Apply f^{test} to the query set from the new test domain.
- Key: No overlap in classes between training tasks and test task (e.g., train on cats/lambs/pigs, test on ducks/dolphins/hens).
- Algorithm M must learn to predict image classes *in general*.
- Many approaches, see [Wan+20b] for a survey.

FSL: Matching Networks [Vin+16] (1/2)

- Approach: Learn a distance metric on some other dataset, then use it in a k-Nearest Neighbor classifier.
- Defines a semi-parametric model $p_{\boldsymbol{\theta}}(y|x, S)$, where S is the small labeled support set.
- $\boldsymbol{\theta}$ are parameters of the distance/similarity function.
- Matching Networks use an adaptive similarity kernel (attention mechanism).
- Attention compares $\boldsymbol{x}$ to $\boldsymbol{x}_n$ in context of all support examples.

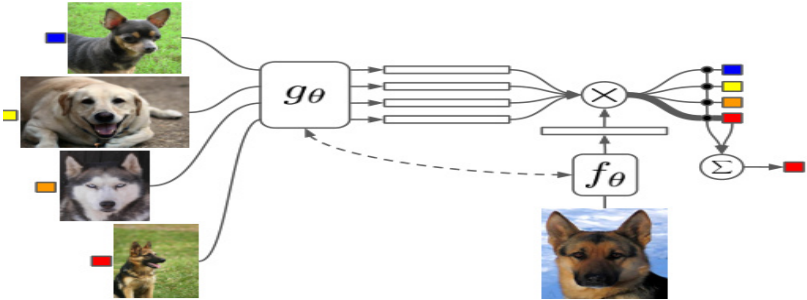


Figure: Matching network for one-shot learning. (Fig 19.16 from source)

FSL: Matching Networks [Vin+16] (2/2)

- Matching Networks prediction rule:

$$p_{\boldsymbol{\theta}}(y|\boldsymbol{x}, S) = \mathbb{I}\left(y = \sum_{n \in S} a_{\boldsymbol{\theta}}(\boldsymbol{x}, \boldsymbol{x}_n; S) y_n\right)$$

- Attention kernel $a_{\boldsymbol{\theta}}$ is computed via softmax over similarities:

$$a(\boldsymbol{x}, \boldsymbol{x}_n; S) = \frac{\exp(c(f(\boldsymbol{x}), g(\boldsymbol{x}_n)))}{\sum_{n'=1}^{N} \exp(c(f(\boldsymbol{x}), g(\boldsymbol{x}_{n'})))}$$

- $c(\cdot, \cdot)$ is cosine similarity; f, g are embedding functions (e.g., LSTMs or CNNs).
- Entire pipeline is differentiable, trained end-to-end to learn f, g, and $\boldsymbol{\theta}$.

Matching Networks: Training

- Train embedding functions f, g (parameters $\boldsymbol{\theta}$) using multiple small datasets (meta-learning setup).
- Let $\mathcal{D}$ be a large labeled dataset (e.g., ImageNet).
- Let $p(\mathcal{L})$ be a distribution over its labels.
- Create a task:
  1. Sample a small set of labels $\mathcal{L}_{task} \sim p(\mathcal{L})$.
  2. Sample a small support set $S \sim \mathcal{L}_{task}$ from $\mathcal{D}$ with these labels.
  3. Sample a small query set (test set) $T \sim \mathcal{L}_{task}$ with the same labels.
- Optimize objective: Maximize log-likelihood of query set labels given support set.

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{D}) = \mathbb{E}_{\mathcal{L}_{task} \sim p(\mathcal{L})} \left[\mathbb{E}_{S \sim \mathcal{L}_{task}, T \sim \mathcal{L}_{task}} \left[\sum_{(\boldsymbol{x}, \boldsymbol{y}) \in \mathcal{T}} \log p_\theta(\boldsymbol{y} | \boldsymbol{x}, S) \right] \right]$$

- After training, freeze $\boldsymbol{\theta}$ and apply to a new test support set.

Weakly Supervised Learning: Introduction

- Refers to scenarios where we do not have an exact label associated with every feature vector in the training set.
- The supervision is "weaker" than in standard supervised learning.
- Different types of weak supervision exist.

Weak Supervision: Learning with Label Distributions

- Scenario: Have a distribution over labels for each case, $p(y|\boldsymbol{x}_n)$, rather than a single label.
- Can still do maximum likelihood training by minimizing cross-entropy:

$$\mathcal{L}(\boldsymbol{\theta}) = -\sum_n \sum_y p(y|\boldsymbol{x}_n) \log q_{\boldsymbol{\theta}}(y|\boldsymbol{x}_n)$$

  where $q_{\boldsymbol{\theta}}(y|\boldsymbol{x}_n)$ is the model's predicted distribution.
- **Label Smoothing**: An example where this is artificially done.
  - Replace exact one-hot labels with a "soft" version.
  - E.g., Put $1-\epsilon$ (e.g., 0.9) mass on the true label, and spread ϵ uniformly over other choices.
  - Useful form of regularization [MKH19].

Weak Supervision: Multi-Instance Learning (MIL)

- Scenario: Have a set (or bag) of instances, $\boldsymbol{x}_n = \{\boldsymbol{x}_{n,1}, \ldots, \boldsymbol{x}_{n,B}\}$.
- Only have a label y_n for the entire bag, not for individual members y_{nb}.
- Common assumption (though others exist):
  - If any member of the bag is positive, the whole bag is labeled positive: $y_n = \bigvee_{b=1}^{B} y_{nb}$.
  - If all members are negative, the entire bag is negative.
  - Don't know which member "caused" a positive bag label.
- Introduced in [DLLP97].
- Example: COVID-19 risk score learning [MKS21].
- Various algorithms exist depending on assumptions about label correlations within bags and expected fraction of positive instances [KF05].

Weak Supervision: Distant Supervision

- Often used to train information extraction systems [Min+09].
- Idea: Have a knowledge base of true facts (e.g., "Married(PersonA, PersonB)").
- Use these facts to label sentences in an unlabeled text corpus.
- Example:
  - Fact: Married(Barack, Michelle).
  - Sentence: "Barack and Michelle invited 100 people to their wedding." $\rightarrow$ Labeled as positive example of "Married" relation.
- Heuristic can create noisy labels / false positives.
  - E.g., "Barack and Michelle went out to dinner" would also be labeled positive for "Married".
- Techniques for handling label noise are relevant here (e.g., Section 10.4 of the book).

Conclusion: Beyond Supervised Learning

- This lecture explored crucial techniques for machine learning when abundant labeled data is not available.
- We covered:
  - **Data Augmentation**: Expanding datasets artificially.
  - **Transfer Learning**: Leveraging knowledge from related tasks (including self-supervised learning).
  - **Semi-Supervised Learning**: Combining labeled and unlabeled data.
  - **Active Learning**: Intelligently selecting data to label.
  - **Meta-Learning**: Learning how to learn new tasks efficiently.
  - **Few-Shot Learning**: Making predictions from very few examples.
  - **Weakly Supervised Learning**: Handling imprecise or aggregate labels.
- These methods are vital for real-world applications where data labeling is a bottleneck, enabling powerful models even with limited supervision.